ATLAS: what needs your hands now

What only a person can do right now, step by step.
Everything else is being written by the agent sessions.

State at 19:50 on 3 October 2026. The Station PC is set up. On the Mac: Xcode 26.1 is selected; no Apple account is signed in to Xcode; there is no Apple Development certificate; no phone has shown as ready yet.

The five jobs

	Job	What it is, and what it blocks
☐	**A Sign in to Xcode**	Blocks every phone build.
☐	**B Prepare the phones**	Blocks the first build and all testing on a phone.
☐	**C First build to the phones**	The first proof that signing and the phones work.
☐	**D Decisions only you can make**	Four decisions. None is urgent this hour.
☐	**E The agent sessions**	Neither branch has reached GitHub yet.

How to read it. Tick a box when a step is done. Grey blocks are copied out of this PDF and used exactly as printed. Unless a step says otherwise, they go into Terminal on the Mac. A small line that says *You should see:* tells you what a good result looks like.

A Sign in to Xcode

Blocks every phone build.

Already done: the command-line tools point at Xcode, and `xcodebuild -version` prints `Xcode 26.1`.

☐ **A1 Open Xcode › Settings (Cmd + ,) › Accounts.** Click + › Apple Account and sign in. A free account gives a Personal Team.

You should see: the account in the list, with "(Personal Team)" after the name.

☐ **A2 Select the team › Manage Certificates... › + › Apple Development.**

You should see: a new line "Apple Development" in the list.

☐ **A3 Read the Team ID and write it down:**

```
security find-certificate -a -c "Apple Development" -p | openssl x509 -noout -subject
```

You should see: a line that contains `OU=` followed by ten characters. Those ten characters are the Team ID.

Free Personal Team rules. One signing owner and one bundle id (`com.winstonthov.atlas`) for the whole team. A build signed by another account cannot install over the existing one. An installed build stops launching after 7 days; reinstalling renews it. Ten new app ids per 7 days. At most three registered devices.

B Prepare the phones

The iPhone 15 Pro Max first, then the iPhone 14 Pro Max. Do every step for each phone.

☐ **B1 Plug the phone into the Mac, unlock it, tap Trust.**

☐ **B2 In Xcode: Window › Devices and Simulators.** Select the phone in the list.

☐ **B3 On the phone: Settings › Privacy & Security › Developer Mode › on.** The phone restarts; unlock it and confirm.

☐ **B4 Back in Xcode, wait until it has finished preparing the phone.** Then check:

```
xcrun devicectl list devices
```

You should see: the phone with the state `available (paired)`. `unavailable` means it is not plugged in. `connected (no DDI)` usually means Developer Mode is off or Xcode has not finished preparing it: repeat B2 and B3.

- [ ] **B5 Write down the phone's iOS version** (Settings › General › About) for `docs/versions.md`.
- [ ] **B6 Put the phone in the headset:** the rear camera and the LiDAR must not be covered.

Note. The Mac also remembers an iPhone 16e. It has no LiDAR and is not one of the plan's phones.

C First build to the phones

Do it once A and B are done and the lanes A and C session reports that the project compiles.

- [ ] **C1 Close the project in the Unity Editor.**
- [ ] **C2 In Terminal, from the repository root.** Replace the X's with the Team ID; it is needed on the first build only:

  ```
  ATLAS_TEAM_ID=XXXXXXXXXX tools/build_ios.sh
  ```

  You should see: the app installed on every connected phone. Logs are in `ATLAS/Build/`.
- [ ] **C3 If the script cannot install:** open `ATLAS/Build/iOS/Unity-iPhone.xcodeproj` in Xcode and press Run.
- [ ] **C4 First launch on each phone,** with a free account: Settings › General › VPN & Device Management › trust the developer profile.
- [ ] **C5 Accept the camera prompt** with the phone in your hand, not in the headset.

Note. The build script and the app code are still uncommitted work of that session. A compile error there is its code mid-change, not your signing.

D Decisions only you can make

Four decisions. None is urgent this hour.

- [ ] **D1 The organisers' rules.** Needed before the contracts freeze (gate G0).
  - [ ] Read the event's rules and FAQ, and the challenge brief in the repository (`documents/Huawei_Custom_Challenge_1_Beyond_Euclid_2016.docx.pdf`), for what they say about work done before the event.
  - [ ] If they do not say, ask the organisers in writing: may code and assets be prepared before the event; what must be declared as prior work; how long a slot is; one judge or a panel; live demo or video; the space, light and power at the venue; whether our own Wi-Fi router is allowed.
  - [ ] Tell the team the answer and write it under open item 15 in section 14 of the master plan.
- [ ] **D2 The headset.** Needed by gate G4.
  - [ ] Find the model name, on the headset or its box.
  - [ ] Get its three lens numbers: the distortion coefficients, the lens separation and the screen-to-lens distance.
  - [ ] Check the fit with both phones and with someone who wears glasses.
  - [ ] Give the model and the numbers to lane A. The lens correction (task **P1-A05**) needs them; without them straight lines bow.

- [ ] **D3 A hosted image-to-3D service,** for when TripoSG fails on the Station. Needed by gate G6.
  - [ ] Start from the result of the TripoSG test on the Station (Part 6 of the Station guide, `documents/ATLAS_Station_Setup.pdf`): whether TripoSG runs on the RTX 5070, and how many seconds one object takes.
  - [ ] Choose the service. For each candidate note its cost, the licence of what it returns, and how long it keeps the photos sent to it.
  - [ ] Create the account and the key, and give the key to lane D. Never commit it.
  - [ ] Tell lane D what a walker must be told before the scan: with a hosted service, crops of the object photos leave the Station.
- [ ] **D4 The non-commercial licences inside the TripoSG stack.** Needed before the Forge's package list is frozen.

  The facts: TripoSG is MIT. `diso` is CC BY-NC 4.0. TripoSG's own background remover (RMBG-1.4) is non-commercial. The default model of `rembg` (`bria-rmbg`) is non-commercial too.
  - [ ] Check whether the event or the challenge restricts non-commercial parts.
  - [ ] Decide: accept them for the entry, or have lane D avoid them.
  - [ ] Either way, have lane D enter all four in the licence ledger.

E The agent sessions

Nothing else to do by hand.

Two sessions are writing code: lanes A and C on the branch `phase1-acd`, and the lung data pipeline on the branch `p2-lung-data`. At 19:50 neither branch was on GitHub: `phase1-acd` had one commit that was not pushed and 173 uncommitted files, and `p2-lung-data` had no commit.

- [ ] **E1 In each of the two sessions, paste this line:**

```
Commit your work to your branch and push the branch to origin. Do not push to main.
```

- [ ] **E2 Check that both branches reached GitHub.** In Terminal, from the repository root:

```
git fetch origin
git branch -r
```

  You should see: `origin/phase1-acd` and `origin/p2-lung-data` in the list.

- [ ] **E3 When a session reports that it has finished,** read its report and have its branch reviewed before it is merged into main.